MODULE MT-41 · APPLICATIONS WEB DISTRIBUÉES

Dossier technique · Validation intermédiaire (séance 4)

SkillSwap · Plateforme de gestion des échanges de compétences

Partie individuelle

Microservice Skill Service

Responsable : [Nom Prénom]

Classe – Groupe : [Classe – Gx]

Année universitaire 2026-2027

1. Description et responsabilités du microservice

1.1 Rôle

Le **Skill Service** gère le catalogue des compétences de SkillSwap (catégories et compétences) ainsi que les compétences déclarées par chaque membre : **proposées** (OFFERED) ou **recherchées** (WANTED), avec un niveau. Il est le seul propriétaire de ces données (contexte borné « Compétences »).

1.2 Responsabilités

- Gérer le catalogue : catégories et compétences, en écriture pour l'ADMIN et en lecture publique.
- Permettre à chaque membre de déclarer, modifier et supprimer ses compétences (propriétaire uniquement).
- Rechercher les compétences par membre, par compétence ou par type, avec pagination.
- Vérifier, pour l'exchange-service, qu'une compétence est bien offerte par un mentor (appel OpenFeign).
- Publier `UserSkillCreated` et `UserSkillDeleted`, et consommer `UserDeleted` émis par le user-service.
- Renvoyer des erreurs claires (400, 404, 409) au format Problem Details.

1.3 Fiche technique

Élément	Choix
Technologie	Spring Boot (Java 17)
Nom dans Eureka	`SKILL-SERVICE`
Port	8082
Base de données	MySQL privée (aucune clé étrangère vers un autre service)
Messagerie	RabbitMQ, exchange topic `skillswap.events` (producteur et consommateur)
Sécurité	JWT Keycloak, rôles USER et ADMIN
Accès	Via la Gateway : `/api/skills/**`, `/api/categories/**`, `/api/user-skills/**`
Documentation	API documentée avec Swagger (OpenAPI)

2. Diagramme de classes

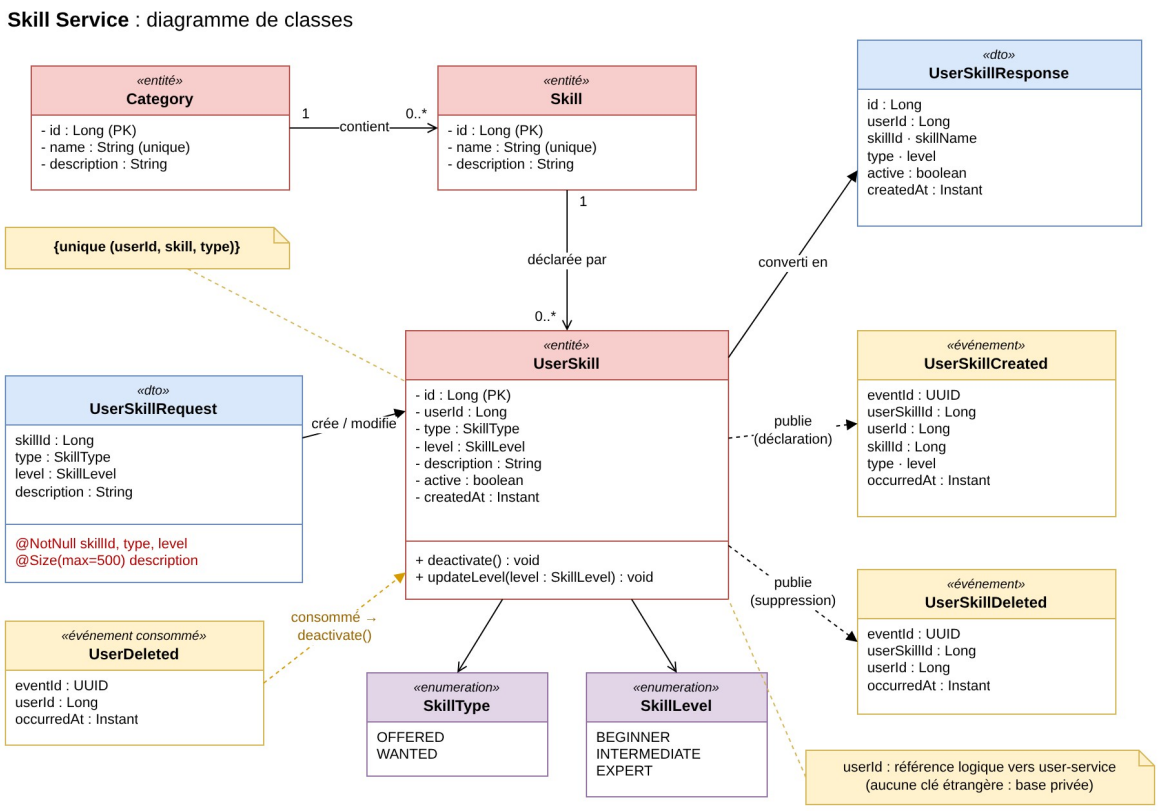


Figure 1 – Diagramme de classes du Skill Service

Le diagramme distingue trois types de classes : les **entités** persistées dans MySQL, les **DTO** échangés avec les clients de l'API, et les **événements** publiés ou consommés via RabbitMQ.

2.1 Entités

Entité	Attributs	Rôle
Category	id (PK), name (unique), description	Domaine de compétences (Informatique, Langues…)
Skill	id (PK), name (unique), description, category	Compétence du catalogue, rattachée à une catégorie
UserSkill	id (PK), userId, skill, type, level, description, active, createdAt	Compétence déclarée par un membre, proposée ou recherchée

`UserSkill` porte deux méthodes métier : `deactivate()`, qui passe `active` à false, et `updateLevel(level)`. Les énumérations `SkillType` (OFFERED, WANTED) et `SkillLevel` (BEGINNER, INTERMEDIATE, EXPERT) typent la déclaration.

> **Point clé :** `userId` est une référence logique vers un membre du user-service, sans clé étrangère. Chaque microservice possède sa propre base ; aucune base n'est partagée.

2.2 DTO et événements

`UserSkillRequest` porte les données saisies par le membre et leurs contraintes de validation (`@NotNull` sur skillId, type et level ; `@Size(max=500)` sur la description). `UserSkillResponse` est la représentation renvoyée par l'API. Les événements `UserSkillCreated` et `UserSkillDeleted` (publiés) et `UserDeleted` (consommé) contiennent un `eventId` qui permet un traitement idempotent.

2.3 Règles de gestion

- Les noms des catégories et des compétences sont uniques.
- Un membre ne déclare qu'une fois une compétence pour un type donné : contrainte {unique (userId, skill, type)}.
- Une catégorie ou une compétence encore utilisée ne peut pas être supprimée (409 Conflict).
- L'`userId` est extrait du jeton JWT, jamais du corps de la requête.

2.4 Choix de la base de données

MySQL, base prévue par la stack pour les microservices individuels : les données sont relationnelles (Category → Skill → UserSkill), soumises à des contraintes d'unicité et d'intégrité, et doivent persister entre les redémarrages.

3. Principaux endpoints REST et codes HTTP

Tous les appels passent par l'API Gateway (port 8080). Le catalogue est consultable sans authentification ; les autres routes exigent un jeton JWT Keycloak.

3.1 Catalogue

Méthode	Endpoint	Description	Codes HTTP
GET	`/api/categories`	Lister les catégories (public)	200
POST	`/api/categories`	Créer une catégorie (ADMIN)	201, 400, 401, 403, 409
DELETE	`/api/categories/{id}`	Supprimer une catégorie (ADMIN)	204, 401, 403, 404, 409
GET	`/api/skills?categoryId=&q=`	Consulter le catalogue (public)	200
GET	`/api/skills/{id}`	Consulter une compétence (public)	200, 404
POST	`/api/skills`	Ajouter au catalogue (ADMIN)	201, 400, 401, 403, 404, 409
PUT	`/api/skills/{id}`	Modifier (ADMIN)	200, 400, 401, 403, 404, 409
DELETE	`/api/skills/{id}`	Supprimer (ADMIN)	204, 401, 403, 404, 409

3.2 Compétences des membres

Méthode	Endpoint	Description	Codes HTTP
POST	`/api/user-skills`	Déclarer une compétence	201, 400, 401, 404, 409
GET	`/api/user-skills/me`	Consulter mes compétences	200, 401
GET	`/api/user-skills/{id}`	Consulter une compétence (appelé par exchange-service)	200, 401, 404
GET	`/api/user-skills?userId=&type=&page=&size=`	Rechercher (paginé)	200, 400, 401
PUT	`/api/user-skills/{id}`	Modifier (propriétaire uniquement)	200, 400, 401, 403, 404, 409
DELETE	`/api/user-skills/{id}`	Supprimer (propriétaire ou ADMIN)	204, 401, 403, 404

Codes : 200 OK · 201 Created · 204 No Content · 400 requête invalide · 401 non authentifié · 403 accès interdit · 404 introuvable · 409 conflit (nom existant ou compétence déjà déclarée). Les erreurs sont renvoyées au format Problem Details, comme dans les autres services de l'équipe, et l'API est documentée avec Swagger.

4. Événements RabbitMQ

Les événements transitent par l'exchange topic `skillswap.events`, avec des files durables et un acquittement manuel.

Événement (clé de routage)	Sens	Données	Effet
`UserSkillCreated` (userskill.created)	Publié	eventId, userSkillId, userId, skillId, type, level, occurredAt	recommendation-service ajoute l'offre ou le besoin au matching
`UserSkillDeleted` (userskill.deleted)	Publié	eventId, userSkillId, userId, occurredAt	recommendation-service retire la compétence du matching
`UserDeleted` (user.deleted)	Consommé (publié par user-service)	eventId, userId, occurredAt	Skill Service appelle `deactivate()` sur toutes les compétences du membre

5. Interactions avec les autres microservices

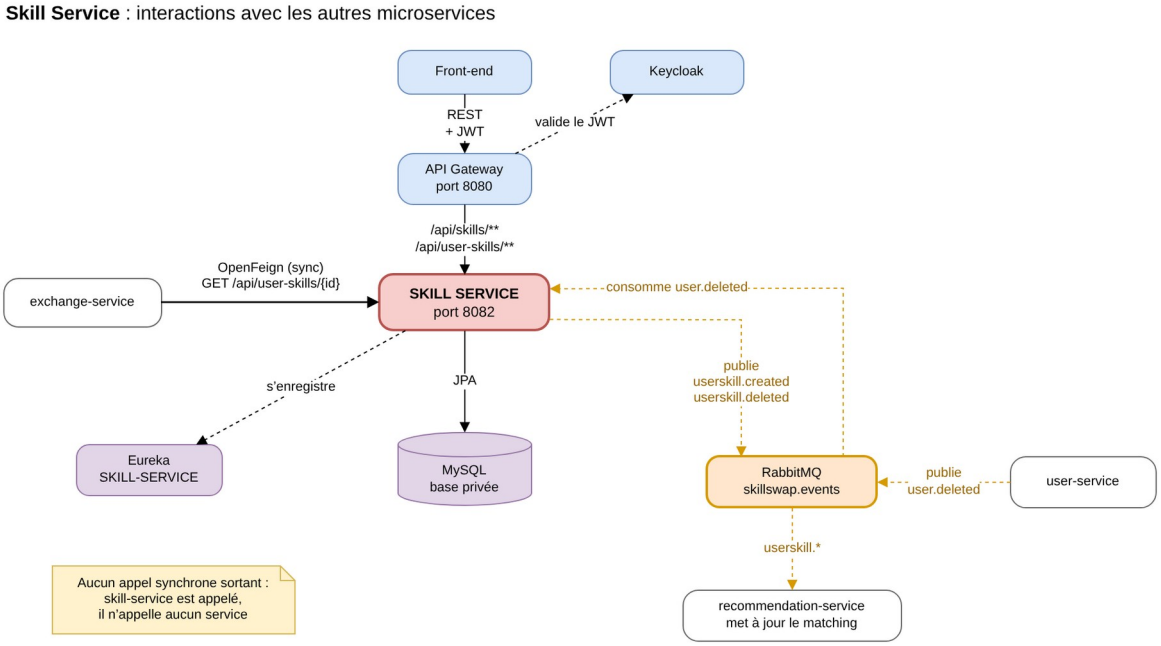


Figure 2 – Interactions du Skill Service

Service	Sens	Mode	Objet
Front-end	Appelle via la Gateway	Synchrone (REST)	Consulter le catalogue, gérer ses compétences
exchange-service	Appelle le Skill Service	Synchrone (OpenFeign)	Vérifier une compétence offerte avant un échange
recommendation-service	Consomme ses événements	Asynchrone (RabbitMQ)	Mettre à jour le matching

Service	Sens	Mode	Objet
user-service	Lui envoie un événement	Asynchrone (RabbitMQ)	Désactiver les compétences d'un membre supprimé

Le Skill Service n'effectue **aucun appel synchrone sortant** : il est appelé mais n'appelle personne. Il reste donc disponible même si les autres services sont arrêtés.

6. Exemple de communication synchrone (OpenFeign)

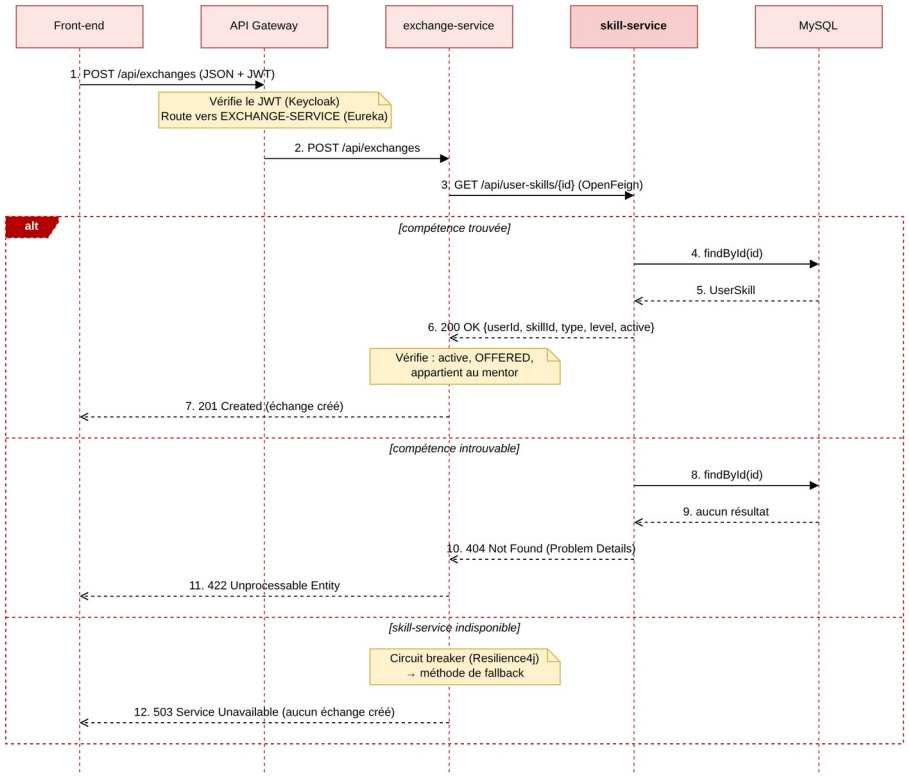


Figure 3 – Vérification d'une compétence par l'exchange-service

Lorsqu'un membre demande un échange, l'exchange-service appelle `GET /api/user-skills/{id}` via un client OpenFeign ; le nom `SKILL-SERVICE` est résolu par Eureka. Le Skill Service renvoie `{ userId, skillId, type, level, active }`, et l'exchange-service vérifie que la compétence est active, de type OFFERED et appartient au mentor.

Justification : la réponse conditionne immédiatement la création de l'échange, d'où le mode synchrone.

Cas d'erreur et de panne : si la compétence est introuvable, le Skill Service renvoie 404 et l'exchange-service refuse la demande avec 422. Si le Skill Service est indisponible, le circuit breaker (Resilience4j) de l'exchange-service déclenche une méthode de fallback et renvoie 503 : aucun échange invalide n'est créé.

7. Exemple de communication asynchrone (RabbitMQ)

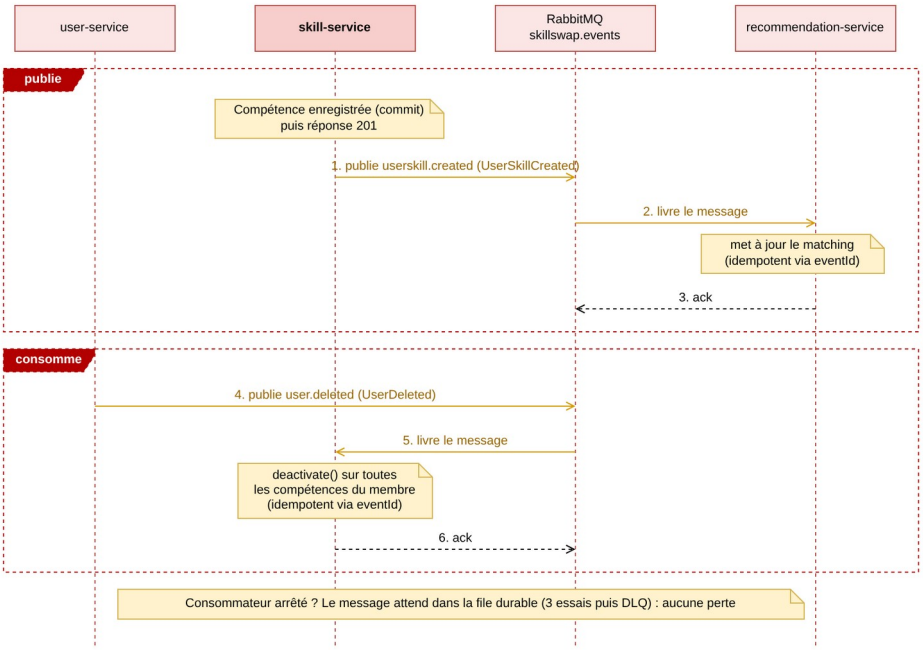


Figure 4 – Événement publié et événement consommé

Publication : après l'enregistrement d'une compétence (commit) et la réponse 201, le Skill Service publie `userskill.created` ; le recommendation-service le reçoit, met à jour le matching puis acquitte le message.

Consommation : lorsqu'un membre est supprimé, le user-service publie `user.deleted` ; le Skill Service désactive toutes les compétences de ce membre puis acquitte. Le traitement est idempotent grâce à l'`eventId` : rejouer le message ne change rien.

Justification : ces traitements sont des effets secondaires ; l'utilisateur n'a pas à les attendre, et les services restent faiblement couplés.

Situation	Comportement prévu
Consommateur arrêté	Le message attend dans la file durable (3 essais puis DLQ) : aucune perte.
RabbitMQ indisponible à la publication	La compétence reste enregistrée en base ; seules les recommandations sont mises à jour en retard.

8. Projet dans le dépôt Git

Le projet Spring Boot est initialisé dans `Backend/skill-service/`, sans logique métier conformément à la consigne « aucun développement avant validation de la conception ». Il compile, démarre et s'enregistre dans Eureka. Les diagrammes draw.io sont placés dans `Documentation/Diagrams/`, et la contribution est visible par des commits personnels réguliers.